Business RAG Knowledge-Base Chatbot

DEMO DOCUMENT - FICTIONAL PRODUCT PRESENTATION. This PDF contains only synthetic, non-confidential content created for testing a Business RAG chatbot. It is safe to upload into a demo environment and must not be treated as real client, legal, financial, or operational advice.

Product summary

The Business RAG Chatbot is a demo website assistant that answers questions from uploaded business documents. It is designed to show how document upload, parsing, retrieval, grounded answers, citations, fallback behavior, logging, and evaluation can work together in a client-realistic AI application.

- Primary user: a website visitor or internal team member who asks questions in natural language.
- Primary admin: a business operator who uploads, deletes, re-indexes, and reviews documents.
- Core value: fast answers from controlled source material instead of unsupported model guesses.
- Demo safety: all examples in this PDF are fictional and safe for testing.

Problem and Product Promise

Business teams often store useful answers inside PDFs, policy pages, onboarding notes, invoices, order ledgers, and support playbooks. A generic chatbot can sound confident without knowing which internal document supports the answer. The Business RAG Chatbot reduces that risk by retrieving relevant document chunks and answering only from the retrieved context.

The product promise

- Answer from uploaded documents, not from unsupported general knowledge.
- Show citations with filename, page number when available, and source snippet.
- Use fallback behavior when the uploaded documents do not contain the answer.
- Give admins visibility into recent questions, retrieved sources, fallback events, and latency.
- Support local testing first, then deployment with managed secrets and persistent storage.

Important failure mode

If retrieval finds weak or unrelated chunks, the chatbot should not invent policy, pricing, legal, HR, billing, or support details. It should say the answer was not found in the uploaded documents and suggest uploading the relevant source material.

Core Workflow

Step	What happens	Demo evidence
1. Upload	Admin uploads TXT, Markdown, or PDF documents.	The UI document panel sends the file to the backend upload route.
2. Parse	Backend extracts text and page metadata where available.	PDF citations can include page numbers.
3. Chunk	Text is split into retrieval-friendly chunks.	Chunks preserve document and page metadata.
4. Embed	The same embedding provider is used for documents and questions.	Similarity scoring ranks candidate chunks.
5. Retrieve	The top chunks are selected across the knowledge base.	The user does not need to pick one document.
6. Answer	The answerer uses retrieved context and returns citations or fallback.	Streamlit displays answer and source expanders.

System Capabilities

Visitor-facing capabilities

- Ask a natural-language question across all indexed demo documents.
- Receive a concise answer grounded in retrieved document text.
- Inspect source snippets in expandable citation cards.
- See a clear fallback warning when the information is not in the uploaded documents.

Admin-facing capabilities

- Upload TXT, Markdown, and PDF files for indexing.
- View document status, chunk count, and filename metadata.
- Delete documents that are no longer part of the demo knowledge base.
- Queue re-indexing when document processing or chunking behavior changes.
- Review recent query logs, citation counts, fallback flags, and retrieved sources.

Evaluation capabilities

- Run document Q&A evaluation cases after retrieval, chunking, or answering changes.
- Track whether fallback was expected and whether fallback was returned.
- Inspect citation snippets to confirm that answers point to the right source evidence.

Architecture Overview

The MVP architecture is intentionally simple. Streamlit provides a lightweight frontend. FastAPI exposes protected document-management and question-answering endpoints. SQLite stores documents, chunks, metadata, usage records, query logs, and retrieved-source logs. The embedding and answer components are replaceable as the project moves toward a production-grade stack.

Layer	Responsibility	Current MVP
Frontend	Upload, list, ask, show citations, show logs.	Streamlit app.
Backend API	Validation, ingestion, retrieval, answering, logging.	FastAPI.
Storage	Documents, chunks, metadata, logs, usage.	SQLite local database.
Retrieval	Rank relevant chunks for each question.	Local embedding and hybrid scoring.
Answering	Generate or select grounded answer from context.	Rule-based MVP answerer, replaceable with an LLM.
Deployment target	Hosted UI, hosted API, persistent DB, managed secrets.	Future milestone.

Grounding, Citations, and Fallback

Grounding means the chatbot should only answer using retrieved source text. A useful answer is not just fluent; it must be supported by uploaded documents. This is especially important for business domains such as support, HR, finance, legal, policy, and operations.

Citation behavior

- Each factual answer should include at least one source citation.
- A citation should include the filename, page number when available, and a matching source snippet.
- Source snippets help admins and users verify whether the answer is actually supported.

Fallback behavior

- If no relevant chunk is found, the assistant should say the answer was not found in the uploaded documents.
- If retrieved chunks are related but incomplete, the assistant should answer only the supported part and clearly state what is missing.
- Fallback questions should be logged so admins can identify gaps in the knowledge base.

Safe fallback copy: I could not find this information in the uploaded documents. Please upload the relevant policy or contact the responsible team so I can answer from a verified source.

Demo Script and Portfolio Value

A strong demo should show the complete loop: upload a document, see it indexed, ask an answerable question, inspect citations, ask an unsupported question, observe fallback, then inspect query logs. This proves the project is not just a chatbot UI; it is a grounded document-answering system with admin visibility.

Portfolio signal

This demo shows practical AI engineering skills: document ingestion, retrieval, grounding, citation design, backend APIs, Streamlit UI, authentication through environment variables, SQLite persistence, evaluation thinking, and observability.

Final safety reminder

All content in this PDF is fictional and safe for testing. Do not replace this notice with real client data in public demos.